Sprint 2 Checkpoint - Track Completed Work

Status: ☑ 100% COMPLETE
Date: 2026-08-29
Last Modified: 2026-08-29

Completed Steps (31/31 Checkpoints ☑)

Phases 1-4: Foundation (10/10 ☑)

- Cloud Logging infrastructure
- Secret Manager integration
- Error standardization (12 error codes)
- Audit logging middleware

Step 5: OpenAPI Documentation (11/11 ☑)

Files Modified:

- `app/models/responses.py` - Response models with Field() descriptors
- `app/models/errors.py` - Error documentation
- `app/api/chat.py` - Endpoint documentation
- `app/main.py` - App-level documentation

Git Commit: `4e46171` - "Sprint 2 Step 5: OpenAPI Documentation Enhancement..."

Step 6: Enhanced Request/Response Logging (10/10 ☑)

Files Modified:

- `app/middleware/audit.py` - Enhanced logging with payload summarization
- `app/utils/metrics.py` - MetricsClient for Cloud Monitoring

Git Commit: Latest - "Sprint 2 Step 6: Enhanced Request/Response Logging..."

Step 7: Cloud Monitoring Metrics (10/10 ☑)

Files Created:

- `app/utils/metrics_definitions.py` - 6 metrics, 5 queries, 4 alert policies
- `app/utils/observability.py` - Context managers and decorators

Git Commit: Latest - "Sprint 2 Step 7: Cloud Monitoring Metrics..."

What Was Changed

New Services Added

- **MetricsClient** (`app/utils/metrics.py`) - Publishes metrics to Cloud Monitoring
- **Observability utilities** (`app/utils/observability.py`) - `track_vertex_latency()`, `track_cache_operation()` context managers

Middleware Enhanced

- **AuditLoggingMiddleware** (`app/middleware/audit.py`):
  - Auth method detection (api_key, oauth2, none)
  - HTTP status text mapping
  - Request payload summarization
  - Response header tracking
  - Metrics recording

Models Enhanced

- All response models have Field() descriptors with examples
- 12 error codes fully documented

Permissions Required (See docs/PERMISSIONS_AND_IAM.md)

Service account: `sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com`

Required roles:

- `roles/monitoring.metricWriter` - Write custom metrics
- `roles/logging.logWriter` - Write to Cloud Logging
- `roles/aiplatform.user` - Call Vertex AI
- `roles/datastore.user` - Access Firestore

Resume Work Later

To continue from here:

1. Check `git log` for latest commit hash
2. All code changes are in `app/` folder
3. Technical details in:
   - Permissions & IAM
   - Session Service
   - Authentication

Next Actions (If Continuing)

```
# 1. Deploy to staging
# 2. Verify Cloud Logging entries appear
# 3. Set up Cloud Monitoring dashboard (use metrics_definitions.py queries)
```

```
# 4. Configure alert policies
# 5. Monitor metrics in production
```

Key Metrics Ready to Deploy

6 custom metrics defined with labels:

1. `api_request_duration_ms` - API latency (SLO: p99 < 2000ms)
2. `api_request_count` - Total requests
3. `api_error_count` - Failed requests (SLO: < 5%)
4. `vertex_api_latency_ms` - Vertex AI performance (SLO: < 30s)
5. `session_cache_operations` - Cache hits/misses (SLO: > 70%)
6. `session_cache_latency_ms` - Cache lookup speed

5 monitoring queries and 4 alert policies included in `app/utils/metrics_definitions.py`.

Zero Regressions ☑

- All endpoints operational
- Error handling unchanged
- Authentication working
- Firestore fallback still works
- All prior phases (1-4) functionality preserved